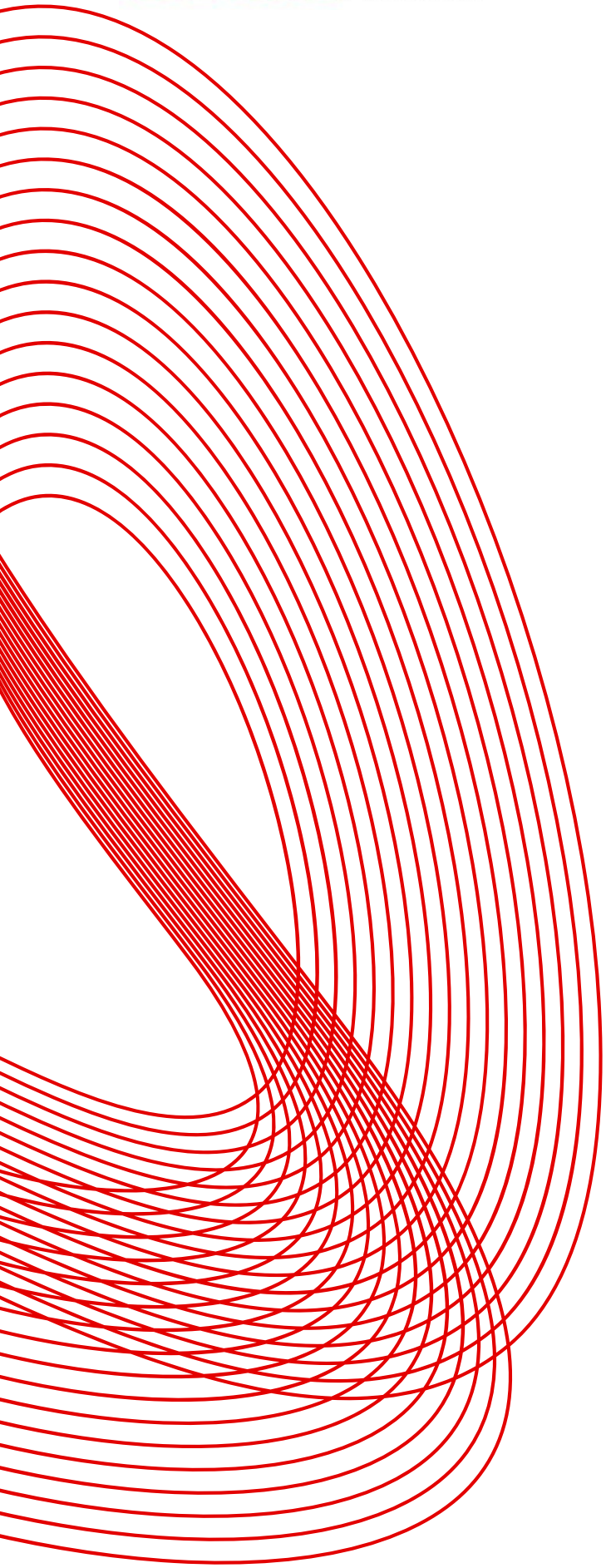
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES STRASBOURG

S9



S-IoT Project Report :

Drone swarm

Written by :

Emilien PEYON

Noé RAMDANI

Class of 2022-2027

Contents

Introduction 4

Constraints 5

I Specifications 6
1 Objective . . . 6
2 Functions . . . 6
3 Deliverables . . . 6

II Bibliographical Studies 7
1 UWB . . . 7
A Definitions . . . 7
B Location algorithm . . . 7
C Timing algorithm . . . 9
2 Measure processing . . . 10
A Polynomial Filter . . . 10
B Extended Kalman Filter for relative localisation . . . 11
C UWB localization : benefits and limits . . . 13
D Data fusion . . . 14

III Main 16
1 UWB . . . 16
A Algorithm . . . 16
B Calibration . . . 16
C Test . . . 16
D Filter . . . 18
2 Communication . . . 20
A Type of communication . . . 20
B Choice . . . 20
C ESP-NOW implementation . . . 21
3 Data processing . . . 22
4 Visualization . . . 23
A Matlab . . . 23
B Architecture . . . 23
5 Simulation . . . 24
A ArduPilot and Mission Planer . . . 24
B Architecture . . . 24
C Result . . . 25
D Simulation possibilities . . . 25
6 Outdoor Test . . . 26

Conclusion 27

Calculations 28

List of Figures

II.1 TWR explanation diagram . . . 7
II.2 DS-TWR explanation diagram . . . 8
II.3 TDMA explanation diagram . . . 9
II.4 Ongoing discrete Kalman filter cycle (Welch and Bishop (2006, p. 5) [**welch2006**]) 12
II.5 "Scheme of the multi-robot system and all onboard sensors" [**li_onboard_2025**] 14

III.1 Schéma de la configuration face à face horizontale . . . 16
III.2 Schema of the paralel configuration . . . 17
III.3 Schema of the paralel configuration . . . 17
III.4 Schema of the paralel configuration . . . 17
III.5 Code for NLOS Filter . . . 18
III.6 Code for NLOS Filter . . . 18
III.7 Code for NLOS Filter . . . 19
III.8 Code for send messsage using ESP-NOW . . . 20
III.9 Drone communication package . . . 21
III.10 Drone communication package . . . 21
III.11 Visualization architecture . . . 23
III.12 Simulation architecture . . . 25

Introduction

Constraints

I - Specifications

1 - Objective

The purpose of this project is to fly a swarm of UAVs (2 to 3 UAVs) and try to lift off a payload using a net.

2 - Functions

- UWB precise location system
- PID Controller for drone stabilization
- UWB Communication for high frequency refreshing data
- Go back and forth keeping the drone formation (following a leader ?)

3 - Deliverables

- Report with all progress, advancements and explanation of technical knowledge
- Functional UWB code
- Methodology and results of calibration of UWB sensors
- Bibliographical Studies
- Video of drone demo
- Functional PID Corrector
- Functional ESP32 code implementation using Multi-threading

II - Bibliographical Studies

1 - UWB

A - Definitions

Ultra-Wideband (UWB) is a technology using radio frequency to exchange data. Communication is not the only purpose; UWB can also be used to locate, which is only possible by using different modules. These algorithms and techniques will be treated in the next part (B). In the majority of cases, UWB is deployed for indoor purposes; otherwise, it can be used for outdoor purposes, but this is less present in scientific papers.

B - Location algorithm

In order to use UWB as a location sensor, we have to use specific algorithms. In the following subsections, we will explain and show 2 different location algorithms: firstly, two-way ranging (TWR), and secondly, double-sided two-way ranging (DS-TWR). Finally, we will describe TDOA, the most advanced algorithm.

Two way ranging (TWR)

The Two-Way Ranging (TWR) algorithm is the easiest and the fastest to implement in a UWB system. In order to explain this algorithm, we will use 2 modules called UWB_1 and UWB_2.

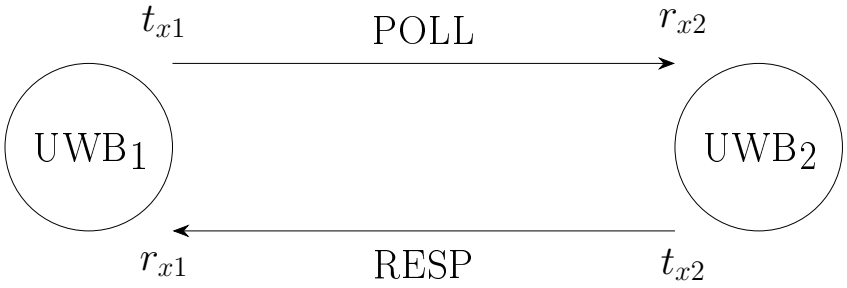


Figure II.1: TWR explanation diagram

The algorithm works in 4 steps:

1. UWB_1 sends a **POLL** to UWB_2. The time of emission t_{x1} is memorized by UWB_1.
2. UWB_2 receives the **POLL** and memorizes the time of arrival r_{x2}.
3. UWB_2 processes the **POLL** and precisely plans the time of emission t_{x2} for the **RESP**. It contains (t_{x2}, r_{x2}).
4. UWB_1 receives the **RESP** and memorizes r_{x1}.

With all these parameters, we can deduce these variables:

- Total time: $t_{tot} = r_{x1} - t_{x1}$
- Processing time of UWB_2: $t_{proc} = r_{x2} - t_{x2}$
- Travel time: $t_{travel} = t_{tot} - t_{proc}$

The signal travels back and forth, so it covers twice the distance (d), and the travel speed of a radio signal is equivalent to the speed of light (c).
We can deduce this equation:

$$\boxed{\frac{t_{travel} \times c}{2} = \frac{(t_{tot} - t_{proc}) \times c}{2} = \frac{((r_{x1} - t_{x1}) - (r_{x2} - t_{x2})) \times c}{2}} \tag{II.1}$$

We can also determine the error induced by this algorithm.
After calculation (6), we obtain this equation:

$$\epsilon \approx \frac{t_{proc}(e_a - e_b)}{2} \tag{II.2}$$

As we can observe, the processing time is directly linked to the error. The t_{proc} is around 3,000,000 nanoseconds, so it induces a high error.

Double Sided Two way ranging (DS-TWR)

In the TWR algorithm, the main inconvenient is the error and misprecision. To erase that error, the DS-TWR has been thinked. This algorithm is also using UWB_1 and UWB_2.

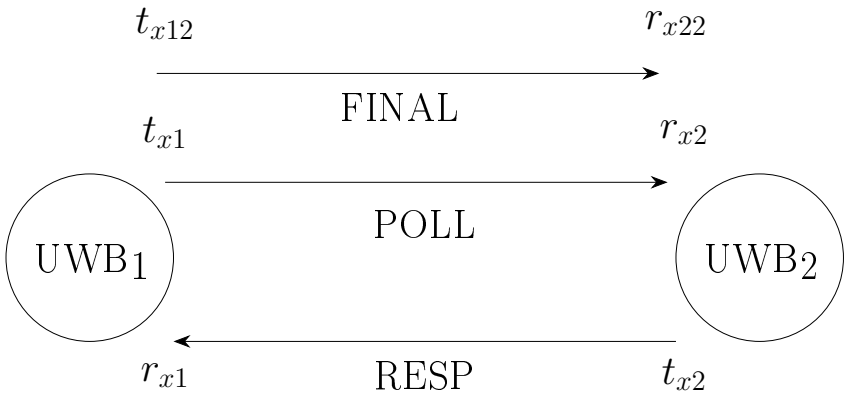


Figure II.2: DS-TWR explanation diagram

The algorithm works in 6 steps :

1. UWB_1 sends a **POLL** to UWB_2. The time of emission t_{x1} is memorized by UWB_1.
2. UWB_2 receives the **POLL** and memorizes the time of arrival r_{x2}.
3. UWB_2 processes the **POLL** and precisely plans the time of emission t_{x2} of the **RESP**, so that the reply delay $D_b = t_{x2} - r_{x2}$ is known.
4. UWB_1 receives the **RESP** and memorizes the time of arrival r_{x1}.
5. UWB_1 processes the **RESP** and precisely plans the time of emission t_{x12} of the **FINAL**, so that the reply delay $D_a = t_{x12} - r_{x1}$ is known. The **FINAL** contains (t_{x1}, r_{x1}, t_{x12}).
6. UWB_2 receives the **FINAL** and memorizes the time of arrival r_{x22}.

With all those parameters, we compute four time intervals :

- Response time for $UWB_1 : t_1 = r_{x1} - t_{x1}$
- Response time for $UWB_2 : t_2 = r_{x22} - t_{x2}$
- Processing time of $UWB_1 : t_{proc_1} = t_{x12} - r_{x1}$

- Processing time of $UWB_2 : t_{proc_2} = r_{x2} - t_{x2}$

We can deduce the travel time:

$$\boxed{t_{travel} = \frac{t_1 \times t_2 - t_{proc_1} \times t_{proc_2}}{t_1 + t_2 + t_{proc_1} + t_{proc_2}}} \tag{II.3}$$

And then find distance :

$$d = c \times t_{travel} \tag{II.4}$$

Now let's compare the induced error. After calculation (6), we obtain this equation :

$$\epsilon \approx T_f \times \frac{e_a + e_b}{2} \tag{II.5}$$

As we can see, the error is not multiplie by t_{proc}, so the error is reduced.
With this algorithm, we are note dependant of the processor any more. That the algorithm currently use in our project.

C - Timing algorithm

When working on swarm drone, there is not only two UWB modules. There is multiple UWB trying to communicate. One weakness of the UWB is it sensibility to noise and interference. The way should be clear before emiting otehrwise that create colision and damaged the singal and corrupt distance estimation. For this purpose, we need to look on how to divide time to avoid collision, in our specifif case, we are working with 3 to 4 drone.

ALOHA

This method is named after the first ALOHAnet system, which introduced it in 1977. It provides a simple approach to channel access control without requiring any time synchronization. Instead of planning transmissions, a module sends its data packet immediately, regardless of whether the medium is currently busy or free. The primary advantage of ALOHA lies in its ease of implementation and full decentralization, allowing independent nodes or drones to join the network dynamically without prior initialization. However, its major drawback is its susceptibility to collisions caused by uncoordinated simultaneous transmissions. As demonstrated in foundational studies, the maximum theoretical throughput (channel capacity) of ALOHA is limited to approximately 18.4% (ref du papier). This low efficiency makes it unsuitable for high-frequency localization and critical communication applications.

Time-division multiple access (TDMA)

This method has for idea to divide a fixed period of time in trame, and cut it in different tranches. Each tranche is for each drone. Those trames and tranches are fixed in the hardware and are stricly respected.

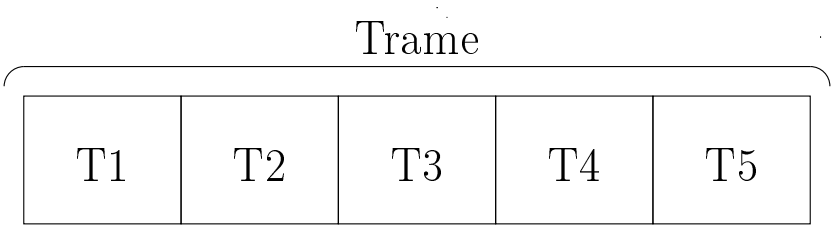


Figure II.3: TDMA explanation diagram

For exemple if we have 3 drone, trame of UWB_1 when is looking for distance in DS-TWR algorithm is divided by 5.

1. T1: UWB_1 send the **POLL** to UWB_2 and UWB_3
2. T2: UWB_2 send the **RESP** to UWB_1
3. T3: UWB_1 send the **FINAL** to UWB_2
4. T4: UWB_3 send the **RESP** to UWB_1
5. T5: UWB_1 send the **FINAL** to UWB_3

This method has an advantage: there is no collision at all due to strict time period. And also permit to shut down the module when we do not need it. But there is inconvenient, firstly if we hade one more drone, we have to add two other tranches, when we reach 10 drone, there is no time anymore to get information at a high frequency. And, it needs a synchronization to make all clock working at same speed and avoid collision perfectly.

Slotted ALOHA

This method combines the features of both the ALOHA and TDMA protocols. Time is divided into tranches, and UWB modules are only allowed to start their transmissions at the beginning of a slot boundary. If a module has data ready to transmit, but a slot has already started, it must wait for the next available slot.

This approach preserves the flexibility of ALOHA while doubling the maximum theoretical throughput to approximately 36.8% (ref du papier). However, it still remains less deterministic and efficient than TDMA due to potential collisions when multiple nodes choose the same slot.

2 - Measure processing

In most cases, data measured by sensors are unclear. It could be a sensor offset, or it can also come from crosstalk or other exterior interactions. In order to obtain clean measurements and to be able to use them for controlling or any other purpose, we have to develop some filters. In this section, we will describe three filters used for cleaning the UWB signal. Firstly, we will explain the polynomial filter, secondly the Kalman filter, and finally the extended Kalman filter.

A - Polynomial Filter

B - Extended Kalman Filter for relative localisation

In the context of navigating systems in noisy environments with sensors, the Kalman filter is a relevant algorithm. As Welch and Bishop (2006, p. 1) [3] point out, the Kalman filter is notable for its ability to "support estimations of past, present, and even future states [...] even when the precise nature of the modeled system is unknown.". In our study case, we deal with a drone swarm communicating using the UWB communication protocol, which gives us the relative distance between each pair of drones by using the time of flight of a radio wave. The UWB therefore provides a scalar distance, whereas the quantity of interest is the relative position vector. The measurement model, which links the two through the Euclidean norm, is nonlinear, which calls for the extended Kalman filter. This version resorts to a typical technique of linearization around the current state using a Jacobian matrix. We are also considering an anchor-free system which enhances the challenge of this project. In this section, we will first cover the theory of the classical Kalman filter and its mathematical formalist and then dive into the extended Kalman filter and its utility.

Theory of the Kalman filter

Origin of the theory In his pioneering article of 1960 [**kalman1960**], Rudolf Kalman describes state estimation as a recursive process in which at each time step, the current estimation and the uncertainty are computed using only the past estimate and the newly available measure. When assuming a linear dynamical system and Gaussian noise, the Kalman filter minimizes the mean-square estimation error : in other words, the best possible estimate given the available measurements. Even when these assumptions do not hold exactly, the filter remains the best linear estimator achievable (known as the Best Linear Unbiased Estimator, or BLUE). This optimality is what established the Kalman filter as the standard approach for state estimation in noisy dynamic systems. While Kalman filter's optimality holds for linear dynamical systems, we have in our case, as mentioned earlier, a non-linearity with the use of the Euclidian distance. This is why we require an extended version of the filter that can handle the non-linearity, even at the cost of losing the strict optimality.

Formalism of the discrete Kalman Filter Let's note $x_k \in \mathbb{R}_n$ the current state of a discrete system governed by the following linear equation :

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$$

In which we have :

- F_k is the state transition matrix, linking the previous state x_{k-1} to the current state x_k
- B_k is the control-input matrix, linking the control vector u_k to the current state x_k
- w_k is the process noise which is assumed to be Gaussian and centered, with a covariance matrix Q_k

At each time k, the measurement z_k is written as follows :

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$$

With :

- H_k is the observation matrix
- w_k is the observation noise which is assumed to be Gaussian and centered, with a covariance matrix R_k

We can divide the Kalman filter process into 2 phases : the "prediction" phase and the "update" phase. The first one does an estimate of the state at the next time step by using the estimate of the current time step and the error covariance. The second phase can be thought of as a feedback : it includes a new measurement in the current state to improve its estimate.

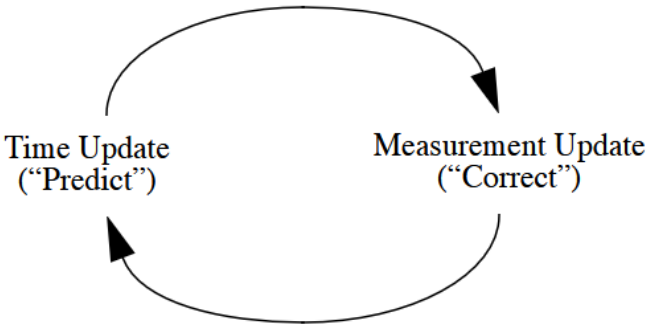


Figure II.4: Ongoing discrete Kalman filter cycle (Welch and Bishop (2006, p. 5) [**welch2006**])

Extended Kalman filter

As mentioned earlier, the extended Kalman filter is used to deal with a non-linear state process defined as diffentiable function f. To do so, it essentially linearizes the nonlinear function around the current estimate by computing the Jacobian matrices of the nonlinear functions f and h in the following equations (with the same notations as before) :

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)$$

$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k)$$

Like the original filter, it is recursive: only the current estimate and its covariance are stored, which keeps memory and computation low enough for embedded platforms, while the covariance provides a measure of uncertainty that can be used to reject inconsistent measurements. However, this comes with a drawback. The linearization is a first-order approximation, so the optimality of the linear Gaussian case is lost, and the filter may diverge if the initial estimate is poor or the system is strongly nonlinear [**boutayeb_convergence_1997**]. Its performance also depends on the tuning of the noise covariances. Alternatives such as the unscented Kalman filter [**julier_new_1997**], particle filters [**arulampalam_tutorial_2002**] or optimization-based estimators differ in accuracy, robustness and computational cost. The EKF nevertheless remains the standard approach in navigation, including in open-source flight controllers (like in the ArduPilot firmware).

C - UWB localization : benefits and limits

As described in the previous section, UWB ranging enables centimeter-level accuracy via time of flight calculations. However, the data gives us only the relative distances between each drone. This data lacks information, particularly about orientation. In this section, we will discuss the viability of a distance-only approach and the topic of observability.

Range-only system

In their paper entitled "Dynamic Range-Only Localization for Multi-Robot Systems", Cao et al. (2018) [**cao_dynamic_2018**] propose a method to perform robot localization via range measurement only. To do so, they present a technique based on successive stages which allows to dynamically localize multiple robots by using trilateration and an EKF recursive estimation. While this article shows a functional multi-agent system with relative localization without anchors, it has been demonstrated only in simulation and in a 2D environment : it does not address any experimental validation nor three-dimensional flight. Moreover, their approach uses a coordinate system built by the robots themselves while drones are usually localized in an Earth-fixed frame via their flight controllers. The authors do acknowledge the question of a global frame but consider it outside the scope of their work, mentioning that a swarm deployed in an unknown environment might no have access to global positioning.

Observability

As defined in [**astrom_feedback_2021**], we say that "a system is *observable* if for every $T > 0$, it is possible to determine the state of the system $x(T)$ through measurements of the $y(t)$ and $u(t)$ in the interval $[0, T]$" where $y(t)$ and $u(t)$ are respectively the output and the input of the process at time t. Observability plays an important role in control theory because it is common that all the states are not accessible through measurement : it is therefore possible to build an *observer*, a dynamical system represented by a mathematical model capable of recreating the states of the system.

In our project, the observability problem is harder to deal with because it would be necessary to know which states of the drones we choose to consider. To get a rough idea of the problem, we can look at the article from Van der helm et al. (2020) [**van_der_helm_-board_2020**] who studed a similar problem : they implemented UltraWide band technology for both ranging and communication between Parrot drones (transmission of the velocities, accelerations, yaw rates and heights). They tested different configurations of leader-follower flights and performed observability analyses. They showed, on the one hand, that without a common heading, "the motion of agents must meet more stringent conditions to be observable and the relative localization becomes more susceptible to noise on the range measurements". On the other hand, the dependency on a common heading offers the advantage of being less sensitive to local disturbances in Earth's magnetic field, which, even when small, can lead to a large negative impact on the EFK used. In a leader-follower configuration, a fixed-geometry (like 2 drones moving in parallel) is not possible and for that, they implement a method where the follower flies in a time-delayed trajectory version of the leader. Their approach was validated indoors with an on-board EKF. Howewer outdoor operations with GNSS (Global Navigation Satellite System) are not addressed.

D - Data fusion

Data fusion with an EFK in drone swarm applications

In the context of our project, the most relevant article is most likely "Onboard Ranging-Based Relative Localization and Stability for Lightweight Aerial Swarms" by Li et al. (2025) [**li_onboard_2025**] from TU Delft in the Netherlands. In their paper, they share an autonomous relative localization technique fusing UWB distance measurements and state information like velocity, yaw rate, height from neighbours. The following illustration shows their scheme of work :

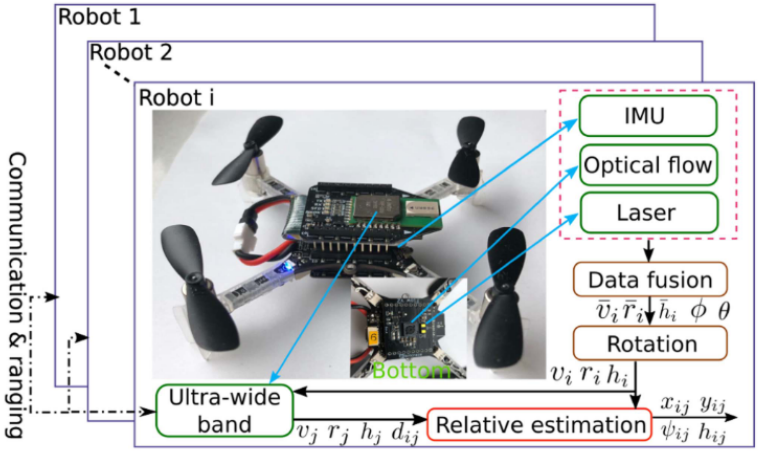


Figure II.5: "Scheme of the multi-robot system and all onboard sensors" [**li_onboard_2025**]

By using the data of 3 sensors (IMU, optical flow, laser), they retrieve the 3-axis velocity $\overline{v_i}$, the pitch θ and the roll ϕ in the body frame of the drone. The yaw rate $\overline{r}$, the range d (distance between 2 drones) and the height h are respectively given by the gyroscope, the UWB sensors and the laser. One of the main aspects of their project is that they work with tiny, lightweight (33 grams) drones which are resource-constrained. Therefore, an Extended Kalman filter is exploited to combine the data, particularly because it has a low computational overhead. The article is particulary appropriate because it deals with every aspect of our project. It brings up the fusion of multiple sensors through an EFK, the use of Ultra-wide band for both ranging and communication as well as observability considerations and hardware resource constraints. Nevertheless, they do not implement GPS when merging data, which differs from the context of our work.

GPS consideration

The papers mentioned previously did not include GPS localization in their architecture but our drones do include such a tracker, which we would like to take advantage of. We are working on an outdoor system and it is possible through the flight controller to use the GPS data to get a more accurate state estimate of the drone. The details about the hardware will be given in the next section.

In an article of 2020, Chen et al. [**chen_uwb-assisted_2020**] studied the complementarity of UWB measurements with GPS localization to get a better accuracy. They showed a great improvement of devices localization with an accuracy up to 10 centimeters. This approach is different from ours as the GPS data is directly combined through a cost function with the UWB measurements while we are planning to use an Extended Kalman filter. Additionally, their experiments were made on static objects and the final localization is obtained in a 2D plane, while

we deal with moving UAVs in a 3D frame, which may add more challenges.

A closer reasearch to our situation was made by Yang et Lee in 2023 [**yang_ultrawideband_2023**] in which they implemented an Extended Kalman Filter using GNSS and INS (Inertial Navigation System) with complementary UWB measurements in a flight controller firmware. They used the flight controller "Pixhawk 4" (the same as us) and successfully carried out a field test which, however, did not include a moving drone formation.

III - Main

1 - UWB

A - Algorithm

Say that we take DS-TWR et TDMA

B - Calibration

Tableau avec les références

C - Test

Face to Face horizontaly

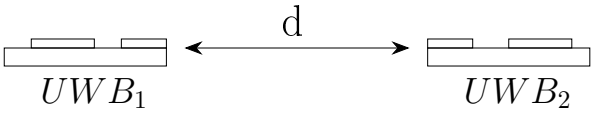


Figure III.1: Schéma de la configuration face à face horizontale

Réelle (cm)	Mesurée (cm)	Erreur (cm)
69,4	61	-8,4
70	62,2	-7,8
75	69,6	-5,4
80	73,9	-6,1
82,6	76,1	-6,5
85	79,9	-5,1
90	86,6	-3,4
91	87	-4
95	92,2	-2,8
100	96,5	-3,5
100	96,8	-3,2
105	102,6	-2,4
107,3	106,4	-0,9
110	108,9	-1,1
115	113,4	-1,6
120,5	120,9	0,4
125	126,4	1,4
130	129,6	-0,4
132	133,7	1,7

Réelle (cm)	Mesurée (cm)	Erreur (cm)
135	133,6	-1,4
140	138,6	-1,4
145	144,9	-0,1
147,4	150	2,6
150	151,7	1,7
155	160,3	5,3
155,5	161	5,5
160	165,7	5,7
164,7	171,8	7,1
165	172,2	7,2
167	172,8	5,8
170	177,2	7,2
175	180,3	5,3
176	183,8	7,8
180	184,3	4,3
185	188,3	3,3
189,6	197,9	8,3
190	191,9	1,9

Table III.1: Mesure face à face de façon horizontale

Paralel module

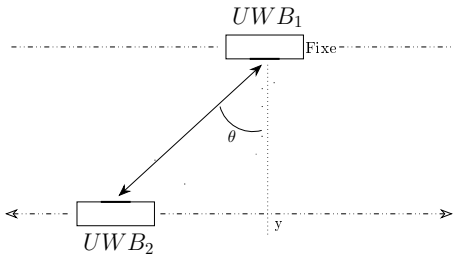


Figure III.2: Schema of the paralel configuration

Décalage (cm)	Angle (°)	Dist. réelle (cm)	Dist. mesurée (cm)	Erreur
0	0	120	117	3
10	4,78	121	118,4	2,6
20	9,59	121,5	118,9	2,6
30	14,48	122,5	119,1	3,4
40	19,47	125,8	124,8	1
50	24,62	128,5	128,7	-0,2
60	30,00	132,5	134,4	-1,9
70	35,69	137	142,6	-5,6

Table III.2: Measure of the paralel configuration

Rayon configuration

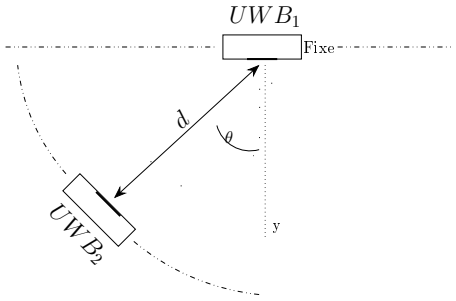


Figure III.3: Schema of the paralel configuration

Angle(°)	Distance réelle (cm)	Distance mesurée (cm)
10	120	116,7
20	120	118,4
30	120	119,5
40	120	122,8
50	120	125,9
60	120	124,2
70	120	123
80	120	124
90	120	130

Table III.3: Measure of the rayon configuration

Face to Back

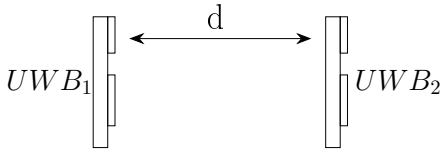


Figure III.4: Schema of the paralel configuration

Distance réelle (cm)	Distance mesurée (cm)	Erreur
70	97	-27
80	109,8	-29,8
90	122,8	-32,8
100	133,3	-33,3
110	143,5	-33,5
120	152	-32
130	161,6	-31,6
140	177,2	-37,2
150	188	-38
160	192,2	-32,2
170	208,8	-38,8

Table III.4: Measure of the rayon configuration

Test de positionement, ajouter les tableaux et les schéma (cote à cote)

D - Filter

Polynomial filter

```c
void polynomial_filter(float* d) {
    float raw_cm = *d * 100.0f;

    // Equation d'erreur = a5*x^4 + a4x^3 + a3*x^2 + a2*x + a1
    const float a5 = -1.004e-6f;
    const float a4 = 0.0005059f;
    const float a3 = -0.09263f;
    const float a2 = 7.398f;
    const float a1 = -221.7;

    float estimated_error_cm = a1 + raw_cm * (a2 + raw_cm * (a3 + raw_cm * (a4 + raw_cm * a5)));

    float corrected_cm = raw_cm - estimated_error_cm;

    if (corrected_cm < 0.0f) {
        corrected_cm = 0.0f;
    }

    *d = corrected_cm / 100.0f;
}
```

Figure III.5: Code for NLOS Filter

NLOS Filter

```c
typedef struct {
    float firstPathPower;
    float receivePower;
    float firstPathAmp1;
} UWB_Diagnostics;

bool NLOS_filter(UWB_Diagnostics diag) {

    float min_fpp_threshold = -85.0f;
    if (diag.firstPathPower < min_fpp_threshold) {
        return false;
    }

    float power_diff = fabsf(diag.receivePower - diag.firstPathPower);
    float max_power_diff = 6.0f;

    if (power_diff > max_power_diff) {
        return false;
    }

    return true;
}
```

Figure III.6: Code for NLOS Filter

Innovation Gate Filter

```
bool innovationGate_filter(float current_d, float prev_d, float max_speed, float dt) {

    float max_possible_delta = max_speed * dt;

    float sensor_noise_margin = 0.2f;
    float allowed_change = max_possible_delta + sensor_noise_margin;

    float actual_change = fabsf(current_d - prev_d);

    if (actual_change > allowed_change) {
        return false;
    }

    return true;
}
```

Figure III.7: Code for NLOS Filter

2 - Communication

In the project, we have to differents use for communication, first communication is used to made each drone talk with each other to maintain the shape but also adapt themselves to the other. Second communication is the data's descent from the drone to the base to permit visualition of the swarm and command drone.

A - Type of communication

We have studied and made research of 3 different communication protocol.

Xbee

The first communication protocol, we make research about is Xbee because it was recommended by the subject. It works well for large range purpose (multiple km range). It has been already implemented by others groups, and by the research team. So, the pipeline was already desin. Otherwise, it needed a supplementary module. Which complexifie the communication management. And, it is in the contrary of the lighweight solution we want to create.

UWB

Second communication protocol was to use the UWB as a sensor and also as communication module. It has been researched and implemented in real life. But during the first test, we have seen some capability reduction due to extension of the message which create a reduction of the frequency. Otherwise, it still a good idea cause it only use the UWB module, that reduce the complexity of the system.

ESP-NOW

The third communication protocol is ESP-NOW. A specific communication protocol made by Espressif (ESP32 manufacturer) for ESP32. It uses the Wi-Fi antenna and it bandwith of 2.4Ghz. Optimize for ESP because code his handled by the manufacturer, and it pass before every other task on the Wi-Fi instruction pile. It is range can reach more than 150m.

B - Choice

We choose ESP-NOW has the protocol for two main raison : it doesn't need new hardware implementation. And, it responds to both communication purpose of the project. Then, has said during the presentation, the code has been optimized and write by the manufacturer which facilitate it implementation. For exemple, send data is only this command :

```
1  esp_now_send(MAC_ADRESS, data, lenght_of_data);
```

Figure III.8: Code for send messsage using ESP-NOW

Has said before, range reach 150 meters allows us to create a base. It only ask us, to have a ESP32, connect to our computer.

C - ESP-NOW implementation

To implement ESP-NOW, we had to think about the data we want to exchange. For our first test, we are using two different packages for two different purpose.
First for inter-drone communication :

```
typedef struct __attribute__((packed)) {
  uint8_t  id;
  uint32_t t_ms;
  float d;
  float z;
  GPS gps;
  Attitude a;
} Pkt;
```

Figure III.9: Drone communication package

And for the base to drone communcation we have this package :

```
typedef struct __attribute__((packed)) {
  uint8_t  id;
  uint32_t t_ms;
  uint8_t commande;
  float variable;
} Pkt_base;
```

Figure III.10: Drone communication package

3 - Data processing

4 - Visualization

Visualization permit to obtain precise measurement and result from test in major case drone will fly outdoor at 10 meters away from us. So, vizualise permit to precisely see the attitude of the drone, the real shape positioning. It can permit to detect error, or problematic point.
To visualise, we choose the sofware Matlab. And had the objectif to be able to see the attitude, the shape position and also look at the data coming from the drone.

A - Matlab

Matlab is a general tool developped by the MathWorks. It has many functions such as mathematics calculator, simulator, and also a corrector designer. Based on add-on and toolbox, it can be usefull for many differents purpose. It also permits to generate an interface.

B - Architecture

The visualization follow the next architecture :

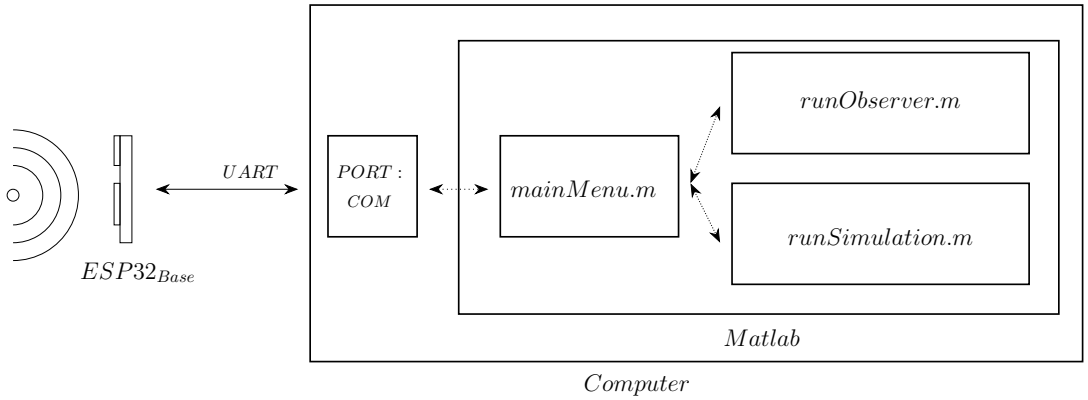


Figure III.11: Visualization architecture

Let's explain each different bloc and its role :

- ESP32 : Relay between ESP-NOW protocol and the UART Port of the computer, which permit to make it visible for the matlab code.
- COM PORT: Physical bridge between the computer and the ESP32
- mainMenu : General window which redirect for the next window
- runObserver : Window to observe UART communication on the PORT (Debug utility)
- runSimulation : Main Window where we visualise attitude of the drone, shape placement and data reader

5 - Simulation

On this project, outdoor tests are very rare due to logistical problem. However, we need to verify our work such as the corrector. To do so, we developed a simulation protocol using the real MAVLink messages, and same data as our drone.
In this part, we will first present the main simulation tools, ArduPilot and Mission planer, second, explain the implementation, then present the result we obtained and finally the simulation possibilities.

A - ArduPilot and Mission Planer

To do drone simulation, we have to select the same architecture as our drone on the pixhwak 4. In the drone industry, two differents architectures are existing: Ardupilot and PX4. In our case, pixhwak 4 is using the ArduPilot architectures.
Then, after selecting the architecture, we have to use a software to visualise and simulate drone. We choose mission planer because we have already work with it on different project.
Let's present both tools:

ArduPilot

ArduPilot was created in 2007 by the DIY Drones community. Originally developed for Arduino boards, it has since become a general-purpose, open-source autopilot. It can control many types of vehicles (copters, planes, rovers, submarines, etc.). ArduPilot provides autopilot functions, receives data from sensors, and fuses it using an Extended Kalman Filter (EKF) [ref]. It also allows users to create pre-planned missions. To operate, it uses the MAVLink protocol to communicate with the drone and facilitate control. In our case, an ESP32 uses the MAVLink library to send MAVLink messages that command the drone's movements.

Mission planer

Mission planer is a representative tool for copters such as drone. Created by Michael Oborne to help users of ArduPilot architecture to vizualise their drone and command them. The software also permits to declare multiple drone so create a swarm. For our project, it will be used as a simulation field.

B - Architecture

The simulation works with a very specific architecture cause tools like ArduPilot works under Linux system. The architecture is exposed in the figure below.

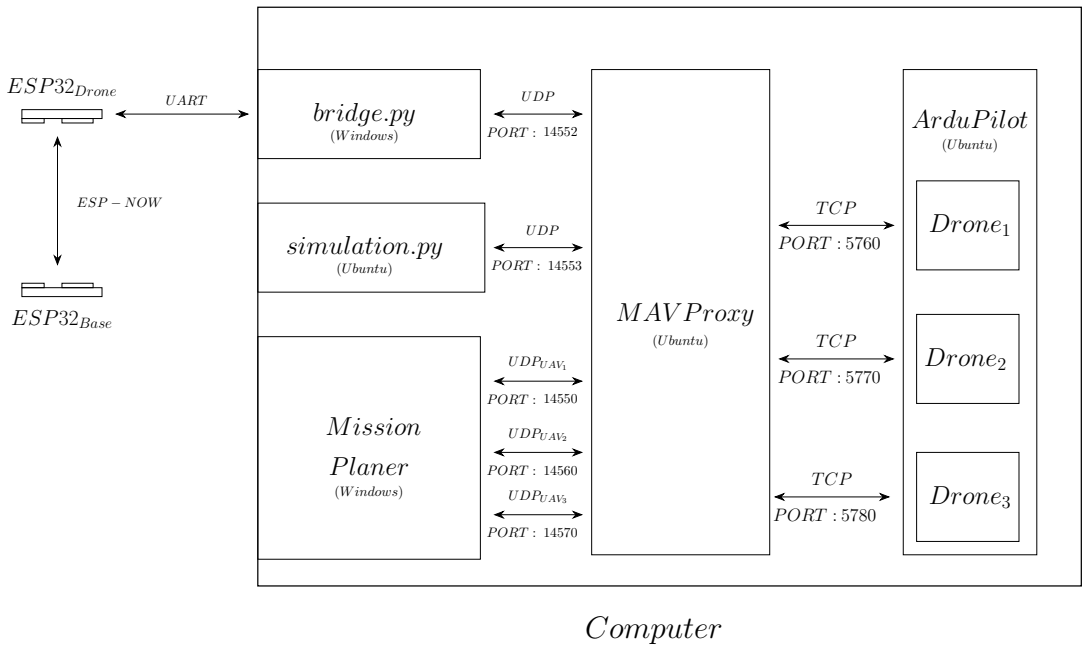


Figure III.12: Simulation architecture

Let's explain each different bloc and its role :

- $ESP - 32_{Base}$: Represent order send from the base for exemple takeoff
- $ESP - 32_{Drone}$: Represent the main drone where code we want to test is running
- ArduPilot : Represent the pixhawk of our drone. Here, we are simulating 3 of them.
- Bridge.py : Code made to redirect data from the UART connection to UDP connection
- MAVProxy : Manage all entries and exit permit to link the drone entries redirect them to the Ardupilot and also permit to redirect exit from ArduPilot (GPS, speed, etc) to Mission Planer.
- Mission Planer : Use to visualise our swarm drone based on data from ArduPilot
- Simulation.py : Python scrip to simulate $Drone_2$ and $Drone_3$

C - Result

Add after fist Test

D - Simulation possibilities

Add after fist Test

6 - Outdoor Test

Conclusion

Calculations

TWR Error calculation :

Let's set:

- True time of flight : T_f
- True time of process : t_{proc}
- Clock drift errors : e_a and e_b

The measured times are:

$$R_a = (2T_f + t_{proc})(1 + e_a) \text{ and } t_{proc_b} = t_{proc}(1 + e_b)$$

The calculated time of flight is:

$$\hat{T}_f = \frac{R_a - t_{proc_b}}{2}$$

By replacing R_a and t_{proc_b}, we obtain :

$$\hat{T}_f = \frac{2T_f(1 + e_a) + t_{proc}(1 + e_a) - t_{proc}(1 + e_b)}{2}$$

By simplifying, we obtain :

$$\hat{T}_f = T_f(1 + e_a) + \frac{t_{proc}(e_a - e_b)}{2}$$

The error is $\hat{T}_f - T_f$, so:

$$Error = T_f \times e_a + \frac{t_{proc}(e_a - e_b)}{2}$$

The time of flight is about 33 nanoseconds, however t_{proc} is around $3,000,000$ns.
So we can ignore $T_f \times e_a$ cause $T_f \ll t_{proc}$, and we obtain :

$$Error \approx \frac{t_{proc}(e_a - e_b)}{2} \tag{III.1}$$

DS-TWR Error calculation :

Let's set:

- True time of flight : T_f
- True times of process : t_{pa} and t_{pb}
- Clock drift errors : e_a and e_b

The measured times are:

$$R_a = (2T_f + t_{pb})(1 + e_a), \quad D_a = t_{pa}(1 + e_a)$$

$$R_b = (2T_f + t_{pa})(1 + e_b), \quad D_b = t_{pb}(1 + e_b)$$

The calculated time of flight is:

$$\hat{T}_f = \frac{R_a R_b - D_a D_b}{R_a + R_b + D_a + D_b}$$

By replacing the measured times, we obtain :

$$\hat{T}_f = \frac{(1+e_a)(1+e_b) \times 2T_f(2T_f + t_{pa} + t_{pb})}{(2+e_a+e_b)(2T_f + t_{pa} + t_{pb})}$$

By simplifying, we obtain :

$$\hat{T}_f = \frac{2T_f(1+e_a)(1+e_b)}{2+e_a+e_b} \approx T_f \left(1 + \frac{e_a+e_b}{2}\right)$$

The error is $\hat{T}_f - T_f$, so:

$$Error \approx T_f \times \frac{e_a+e_b}{2} \tag{III.2}$$